


Model Context Protocol (MCP)

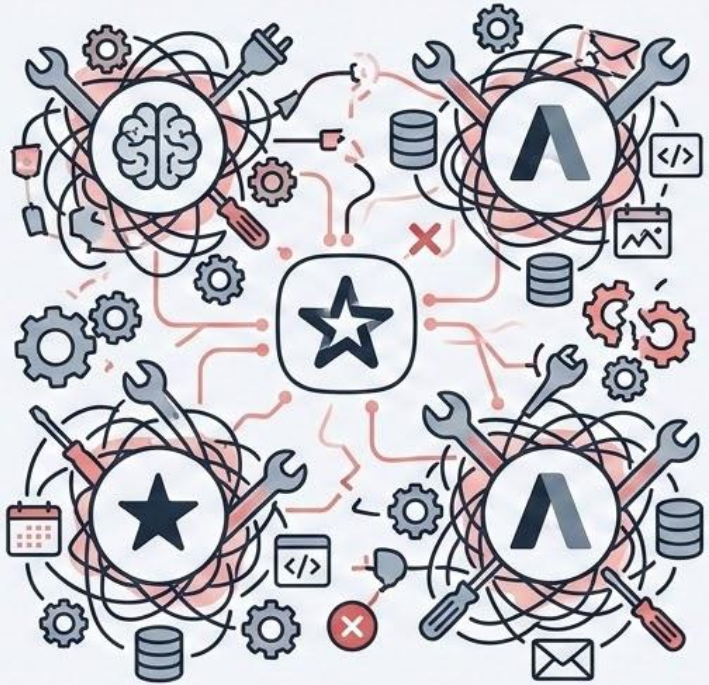
Universal Tools for the Future of Agents


Topics to be Covered


- The Tool Fragmentation Problem
- MCP Architecture
- Using & Building MCP Servers
- Transports & Integration




The Tool Fragmentation Problem



- **Continuous Rewriting**: New integrations require rebuilding tools from scratch.
- **Compatibility Nightmare**: Managing brittle, custom connections across platforms.
- **Maintenance Hell**: High development effort and risk with every API update.
- **Scalability Blocker**: Inhibits the seamless deployment of agents globally.
- **Hidden Costs**: Drains resources on infrastructure instead of innovation.

What is MCP & Why It Matters

MCP (Model Context Protocol) is the standardized universal protocol for integrating AI agents with all external tools and data.

Universal Standard
Consistent connection schema for any agent or tool.

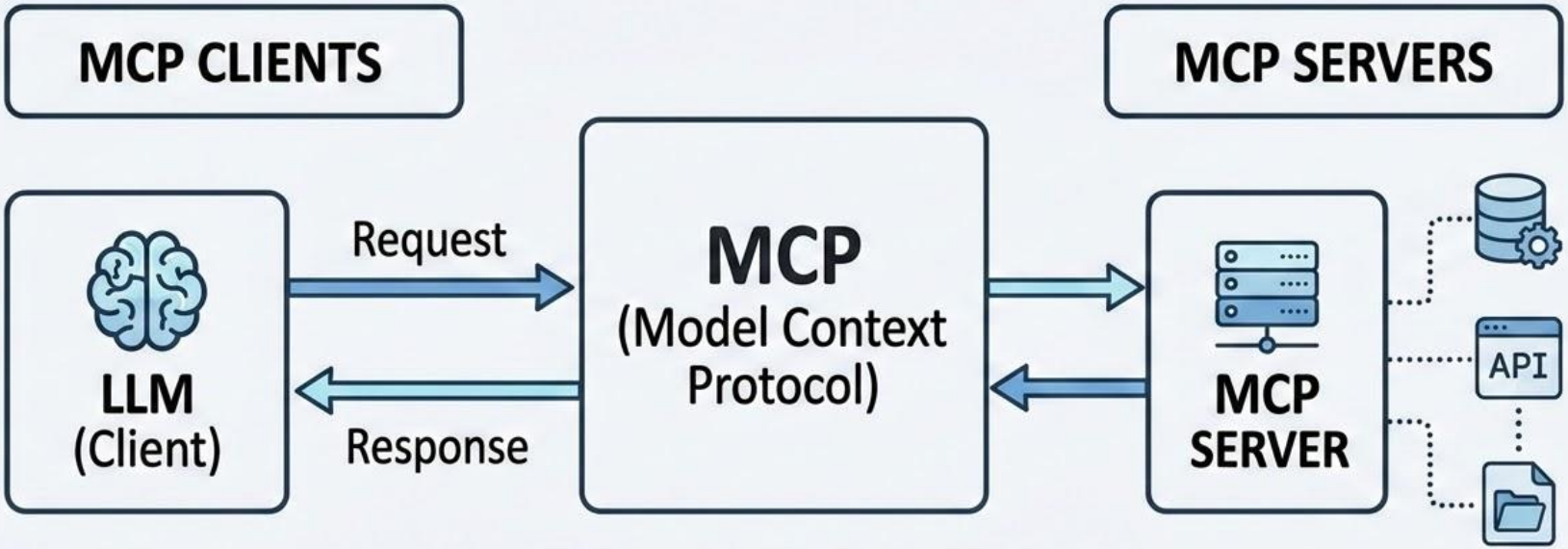
Future-Proof
Updates are managed centrally at the connector, agents from API changes.

Write Once - Use Anywhere
Build tools once and deploy seamlessly across platforms.

Why MCP is Essential:

- ✔ **Eliminate Integration Waste:** End constant rebuilding with reusable tool definitions.
- ✔ **Simplify Your Architecture:** Replace complex custom code with a single, predictable protocol.
- ✔ **Unlock Rapid Scalability:** Enable frictionless global deployment of AI agents.
- ✔ **Focus on Innovation, Not Maintenance:** Direct resources to core features, not API management.

MCP Architecture Overview



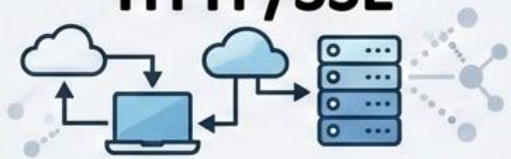
MCP Transports: stdio vs HTTP

stdio

Local, Simple

- **Pros:**
  - Minimal latency
  - High performance
  - Zero network config
- **Cons:**
  - Single machine constraint
  - Limited to one client

HTTP/SSE



Remote, Scalable

- **Pros:**
  - Enable remote agents
  - Multiple simultaneous clients
  - Easily scalable
- **Cons:**
  - Network latency overhead
  - Complex infrastructure

Using Existing MCP Servers

Filesystem

- Access and manage local file systems.
- Read, write, and list directories.

GitHub

- Interact with repositories.
- Manage issues, PRs, and commits.

Slack

- Send and receive messages.
- Search channel and DM history.

Browser

- Navigate and search the web.
- Extract content and interact with websites.

Building Your Own MCP Server

1 Define Capabilities

- Define resources (static/dynamic data)
- Define tools (executable functions)
- Define prompts (templates)

2 Implement Tools & Logic

- Code handlers for tools and resources
- Design input schemas (JSON Schema)
- Write core business logic

3 Select Transport

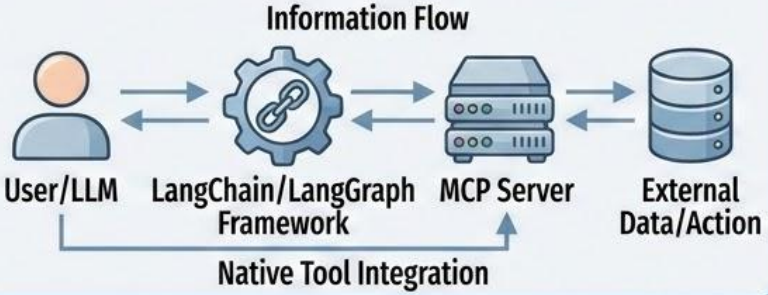
- Choose connection method:
  - Stdio (Local agent)
  - HTTP/SSE (Remote agent)

Code Example: Tool Registration

```
{
  "name": "get_weather",
  "description": "Retrieve current weather",
  "inputSchema": {
    "type": "object",
    "properties": {
      "location": { "type": "string" }
  }
}
```

MCP + LangChain Integration

Conceptual Flow: Agent to Server



Python Code: Connect MCP Server

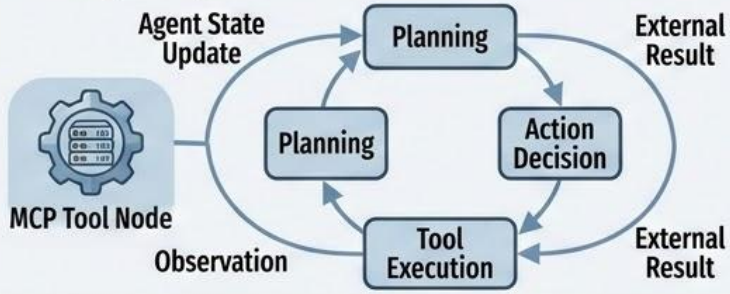
```python
import langgraph
import langchain_mcp

# Connect to local MCP filesystem server
filesystem_server = langchain_mcp.connect("local_filesystem")

# Create a tool from the MCP filesystem server
read_file_tool = filesystem_server.get_tool("read_file")

# Initialize LangGraph agent with the tool
agent = langgraph.create_react_agent(llm, [read_file_tool])
```

LangGraph Interaction: MCP as Tool



Key Advantages: Enhanced Framework

1. **Direct Tool Integration:** Consume MCP functions as native LangChain tools.
2. **Dynamic Capability:** Ground LLMs in proprietary data and external systems without rebuilding APIs.
3. **Framework Synergy:** Leverage the best of both architectures (MCP for tools, LangGraph for orchestration).
4. **Maintainability & Scale:** Centralize tool definitions on mature, secure MCP servers.

Security & Best Practices

Authentication

Verify identity and enforce access control.

- Implement API keys or JWT
- Use role-based access control (RBAC)

Sandboxing

Isolate server operations to limit risk.

- Run MCP server in container (e.g., Docker)
- Restrict filesystem & network access

Error Handling

Handle failures gracefully & avoid data leaks.

- Sanitize error messages (no stack traces to client)
- Implement retry logic where appropriate

Rate Limiting

Protect server resources and prevent abuse.

- Define request limits per client/API key
- Implement DDoS mitigation strategies

Traditional Tools vs MCP

Traditional Integration

- Manual Rebuilding of APIs
- Fragile, Hardcoded Connections
- Siloed Tool Definitions
- Limited Security Implementation

MCP Standardized Framework

- **Dynamic Tool Discovery:** (Highlight Advantage) Ground LLMs without rebuilding APIs.
- **Standardized Access:** Consume MCP functions as native tools.
- **Maintainable & Scalable:** Centralize tool definitions on mature, secure servers.
- **Integrated Security:** Use built-in Authentication, Sandboxing, and Rate Limiting.

Summary

Unified Framework: Seamlessly integrates MCP servers as native tools within LangChain/LangGraph.

Dynamic Capability: Grounds agents in proprietary data and external actions without API rebuilds.

Robust Security: Prioritizes authentication, sandboxing, error handling, and rate limiting.